

Asynchronous & Event-Driven Systems - Mentorship Track

Goal: build strong intuition and interview-ready depth for asynchronous architectures, queues, streams, delivery semantics, and failure handling.

How We Will Use This Sheet

- We will keep this sheet focused on 1.7 Asynchronous & Event-Driven Systems.
 - We will follow the same learning style used in the communication-model notes.
 - We will add topics one by one in a repeatable architect-level structure.
 - We will include code samples, mini programs, and interview-style answers.
-

Roadmap for This Sheet

1. Message queues vs event streams
 2. At-most-once vs at-least-once vs exactly-once
 3. Idempotent consumers
 4. Dead-letter queues
 5. Backpressure handling
 6. Event-driven architecture
-

Topic 1: Message Queues vs Event Streams

Track: 1.7 Asynchronous & Event-Driven Systems Scope: task distribution, fan-out, retention, replay, ordering, consumer groups

1. Intuition

Think of a queue like a work inbox.

One producer drops work into the inbox, and one worker takes each task and finishes it. Once the task is done, it is acknowledged and disappears from the inbox.

Think of an event stream like a durable activity log.

Events are appended in order, and many different readers can look at the same history independently.

One consumer may read for analytics, another for notifications, another for audit, and none of them block each other.

Short memory trick:

- Queue = "someone should do this work"
 - Stream = "this event happened; interested systems may react"
-

2. Definition

- Definition: A message queue is an asynchronous work distribution system where producers send messages for consumers to process, usually with the expectation that each message is handled by one consumer worker.
 - Definition: An event stream is an append-only ordered log of events that multiple independent consumers can read, replay, and process at their own pace.
 - Category: Asynchronous communication and decoupling patterns
 - Core idea: Queues optimize for work dispatch; streams optimize for durable event distribution and replayable history.
-

3. Why It Exists

Synchronous systems couple producer speed to consumer speed.

Queues and streams exist because:

- producers and consumers should not need to be online at the same time
- burst traffic should be absorbed safely
- downstream work should be decoupled from user-facing requests
- many systems need retries, buffering, replay, and independent consumers

The distinction exists because not all asynchronous workloads are the same:

- some workloads are tasks that should be processed once by one worker
 - some workloads are events that many services need to observe independently
-

4. Reality

Message queues are common in:

- email sending
- image or video processing jobs
- background invoice generation
- payment retry workflows
- order fulfillment task dispatch

-
- worker pools and job scheduling

Typical examples:

- Amazon SQS
- RabbitMQ
- ActiveMQ

Event streams are common in:

- order-created and payment-completed event fan-out
- analytics pipelines
- clickstream ingestion
- CDC and data replication
- audit logs
- event sourcing architectures

Typical examples:

- Apache Kafka
- Apache Pulsar
- Amazon Kinesis

Real-world architecture truth

Strong systems often use both:

- a stream for domain events and fan-out
 - a queue for worker-specific background jobs and retries
-

5. How It Works

Message queue flow

1. Producer sends a task message to a queue.
2. The queue stores the message until a consumer is ready.
3. One worker receives the message.
4. The worker processes the task.
5. If successful, the worker acknowledges the message.
6. If the worker fails or times out, the message may be retried or sent to a dead-letter queue.

Key property:

- the message is usually meant to be completed by one consumer path, not replayed forever by many independent readers.

Event stream flow

1. Producer appends an event to a stream or topic.
2. The broker persists the event in order within a partition.
3. Consumers read by offset from the stream.
4. Each consumer group tracks its own progress independently.
5. Events remain retained for a configured period or size.
6. Consumers can replay older events by resetting offsets.

Key property:

- multiple consumers can independently read the same event history.

One-line contrast

- Queue = deliver work to a worker
 - Stream = publish durable events to many readers
-

6. What Problem It Solves

- Primary problem solved by queues: decoupled background work distribution with buffering and retry
 - Primary problem solved by streams: scalable event fan-out with retention, ordering, and replay
 - Secondary benefits: burst absorption, failure isolation, async scalability, reduced request latency on the hot path
 - Systems impact: affects throughput smoothing, consumer independence, recovery strategy, and architecture style
-

7. When to Rely on It

Use a message queue when:

- one worker or one consumer path should process the task
- the message represents work to be completed
- retries and DLQ behavior are important
- you want simple async offloading from request-response flows

Use an event stream when:

- multiple independent consumers need the same event
- replay matters
- ordering matters within a key or partition
- you want event history retained for some time
- analytics, CDC, audit, or fan-out are first-class needs

Strong interviewer keywords for queues:

- background jobs

- task dispatch
- retryable work
- worker pool
- asynchronous processing after API return

Strong interviewer keywords for streams:

- fan-out
- replay
- consumer groups
- retention
- analytics
- event-driven microservices

8. When Not to Use It

Avoid a queue when:

- many systems need to read the same message independently
- you need durable replayable history
- you need long-lived ordered event logs

Avoid a stream when:

- all you need is simple job dispatch to one worker pool
- the team does not need replay, multiple consumers, or log-style retention
- operational simplicity matters more than stream semantics

Better alternatives:

- request-response for short synchronous work
- database state change with polling only for very small/simple systems
- webhooks for server-to-server notification when you do not want to host shared async infrastructure

9. Pros and Cons

Pattern	Pros	Cons
Message queue	Simple async offloading, retries, worker-pool friendly	Weak fit for multi-consumer replayable event history
Event stream	Great fan-out, retention, replay, consumer independence	More operational complexity and partition/offset thinking

10. Trade-offs and Common Mistakes

Trade-offs

- Simplicity vs capability: Queues are simpler for task execution; streams are more powerful for event distribution.
- Single-consumer work vs multi-consumer fan-out: Queues shine when one worker path should act; streams shine when many systems must react.
- Short-lived message handling vs durable history: Queues focus on delivery and acknowledgment; streams preserve log-style history.
- Operational ease vs replay/debug power: Streams are harder to run well, but they give stronger replay and auditing capabilities.

Common Mistakes

Mistake	Why it is wrong	Better approach
Using a queue for many downstream consumers	Creates awkward duplication or extra fan-out layers	Use an event stream or pub/sub style backbone
Using a stream for simple email-job dispatch	Adds unnecessary partition and offset complexity	Use a queue
Assuming "exactly once" end to end without idempotency	Distributed retries and failures still create duplicates	Design consumers to be idempotent
Ignoring partition keys in streams	Leads to hot partitions and broken ordering assumptions	Choose keys intentionally
Treating the broker like a database of record	Brokers are not a substitute for proper state ownership	Persist business state separately when needed

11. Key Numbers

These are practical heuristics and not universal laws.

- Queue backlog: queue depth and age of oldest message are top operational signals
- Visibility timeout: usually longer than expected processing time, often 2-10x the median task duration
- DLQ threshold: often after 3-5 failed processing attempts, depending on task criticality
- Stream retention: often days to weeks, depending on replay and audit needs
- Consumer lag: lag is the core stream health signal
- Partition count: stream throughput and parallelism usually scale with partitions
- Ordering rule: strong ordering is generally per partition, not across the entire stream

Interview shorthand:

- queues care about backlog and retries
 - streams care about lag, partitions, retention, and replay
-

12. Failure Modes

Poison message in a queue

Problem:

- A message always fails processing because of malformed payload or logic error.

User impact:

- repeated retries, worker waste, growing backlog

Mitigation:

- DLQ
- retry limits
- payload validation

Consumer lag in a stream

Problem:

- Consumers fall behind while producers keep appending.

User impact:

- stale analytics, delayed notifications, downstream SLAs missed

Mitigation:

- scale consumers
- increase partitions where appropriate
- optimize consumer processing

Duplicate delivery

Problem:

- A consumer processes a message or event, crashes before committing ack/offset, and receives it again.

User impact:

- duplicate emails, duplicate state changes, duplicate side effects

Mitigation:

- idempotent consumers
- dedupe keys
- transactional patterns where appropriate

Hot partition in a stream

Problem:

- One partition key receives most traffic.

User impact:

- uneven throughput, lag concentrated on one partition, reduced parallelism

Mitigation:

- better partition-key choice
- sharding hot keys
- workload-aware partition design

13. Scenario

- Product / system: E-commerce platform
- Queue use case: After an order is placed, generate invoice PDF, send email, and trigger thumbnail generation for order assets through worker queues.
- Stream use case: Publish `OrderCreated` events so inventory, analytics, fraud detection, notifications, and audit consumers can all react independently.
- Why this split is good: The queue handles concrete tasks; the stream distributes business events to many domains.
- What would go wrong without this distinction: A queue would make multi-consumer fan-out awkward, while a stream would overcomplicate simple job execution.

14. Code Sample

Queue-style worker pseudocode

```
public void handleQueueMessage(OrderEmailJob job) {
    try {
        emailService.sendOrderConfirmation(job.orderId(), job.email());
        queueClient.ack(job.messageId());
    } catch (TransientException ex) {
        queueClient.retryLater(job.messageId());
    } catch (Exception ex) {
        queueClient.moveToDeadLetterQueue(job.messageId());
    }
}
```

Stream-style consumer pseudocode

```
public void onOrderCreated(EventRecord record) {
    OrderCreatedEvent event = deserialize(record.value(), OrderCreatedEvent.class);
}
```



```
analyticsService.recordOrder(event);
notificationService.prepareOrderNotification(event);

streamClient.commitOffset(record.partition(), record.offset());
}
```

Stream producer example

```
public void publishOrderCreated(Order order) {
    OrderCreatedEvent event = new OrderCreatedEvent(order.id(), order.userId(),
order.total());
    streamClient.publish("order-events", order.id(), serialize(event));
}
```

Key idea:

- Queue code is centered on ack, retry, and DLQ.
- Stream code is centered on partitions, offsets, replay, and multiple consumer groups.

15. Mini Program / Simulation

This mini program shows the behavioral difference:

- queue messages are consumed once from the shared work list
- stream events stay in the log and each consumer keeps its own offset

```
from collections import deque

def queue_demo() -> None:
    queue = deque(["job-1", "job-2", "job-3"])
    worker_a = queue.popleft()
    worker_b = queue.popleft()

    print("Queue demo")
    print("Worker A got:", worker_a)
    print("Worker B got:", worker_b)
    print("Remaining queue:", list(queue))

def stream_demo() -> None:
    stream = ["event-1", "event-2", "event-3"]
    offsets = {
        "analytics-consumer": 0,
        "notification-consumer": 0,
    }

    print("\nStream demo")
    for consumer, offset in offsets.items():
        print(consumer, "reads:", stream[offset:])

def main() -> None:
```

```
queue_demo()
stream_demo()

if __name__ == "__main__":
    main()
```

What this demonstrates:

- queue work is shared among workers
- stream history is independently readable by multiple consumers
- queue semantics and stream semantics solve different problems

16. Practical Question

You are designing an order platform. After checkout, the system must send email, update inventory, feed analytics, trigger fraud checks, and persist an audit trail. Would you use a message queue, an event stream, or both? Explain where each fits and why.

17. Strong Answer

I would use both, but for different responsibilities.

I would publish domain events such as `OrderCreated` to an event stream because multiple independent consumers need to react: analytics, audit, fraud detection, notifications, and potentially more services in the future. A stream gives me fan-out, retention, consumer independence, and replay if I need to rebuild downstream state.

For concrete background jobs such as sending an email or generating a document, I would use a message queue because those are task-oriented workloads where one worker path should process the item, with clear retry and DLQ handling.

The key trade-off is that queues are simpler and excellent for work dispatch, while streams are stronger for durable event propagation across many services. I would avoid forcing everything into one model. In practice, strong architectures often use a stream for domain events and queues for worker execution.

18. Revision Notes

- One-line summary: Use queues for work that should be done, and streams for events that should be known and replayed by many consumers.
 - Three keywords: dispatch, fan-out, replay
 - One interview trap: treating queues and streams as interchangeable
 - One memory trick: queue = inbox, stream = history
-

Topic 2: At-most-once vs At-least-once vs Exactly-once

Track: 1.7 Asynchronous & Event-Driven Systems Scope: delivery guarantees, retries, acknowledgments, duplicates, transactional processing

1. Intuition

Think of package delivery guarantees.

- At-most-once: the package is sent once and never retried. It may arrive, or it may be lost, but it should not arrive twice.
- At-least-once: the sender keeps retrying until it is sure the package arrived. This reduces loss, but the receiver may get duplicates.
- Exactly-once: the business outcome should happen once, even across retries and failures.

That last one sounds perfect, but it is also the most expensive and the most misunderstood.

Short memory trick:

- at-most-once = maybe lost, not duplicated
- at-least-once = not lost easily, maybe duplicated
- exactly-once = one effective outcome, but only with strong coordination

2. Definition

- Definition: Delivery semantics describe what a broker and consumer system can guarantee about message delivery under retries and failures.
- Category: Reliability and correctness semantics in asynchronous systems
- Core idea: Failures force a trade-off between message loss, duplication, and coordination complexity.

More specifically:

- At-most-once: a message is delivered zero or one time.
- At-least-once: a message is delivered one or more times.
- Exactly-once: the message's effect is observed exactly once within a defined system boundary.

3. Why It Exists

Distributed systems fail in the middle of work.

Messages can be:

- produced and never persisted

read and processed but not acknowledged

- acknowledged too early
- written to a downstream system before a crash

So systems need explicit semantics for what they optimize:

- lower latency and simplicity
- lower loss risk
- stronger correctness

This topic exists because you cannot eliminate all failure windows for free. You must choose the failure mode you can tolerate.

4. Reality

At-most-once shows up in:

- fire-and-forget telemetry
- low-value monitoring events
- best-effort notifications
- systems where dropping some events is acceptable

Why teams choose it:

- simple
- low overhead
- no retry storms

At-least-once shows up in:

- most production queues
- stream consumers by default
- order/event processing
- email jobs
- asynchronous workflows where loss is unacceptable

Why teams choose it:

- practical default
- easier than exactly-once
- acceptable when consumers are idempotent

Exactly-once shows up in:

- carefully controlled stream-processing pipelines
- financial or ledger-like internal workflows
- systems using transactional processing with offset/state coordination

Why teams choose it:

- they need stronger correctness inside a narrow boundary

Real-world truth:

- most real systems are at-least-once plus idempotent consumers
 - "exactly-once" is usually limited to a specific boundary, not the entire business world
-

5. How It Works

At-most-once flow

1. Producer sends a message.
2. Consumer receives it.
3. The system marks it as delivered or moves on without retry guarantees.
4. If processing fails after that point, the message may be lost.

Typical way this happens:

- auto-commit offsets before processing
- acknowledge too early
- do not retry failed delivery

At-least-once flow

1. Producer sends a message.
2. Broker persists it.
3. Consumer reads and processes the message.
4. Consumer acknowledges or commits only after successful processing.
5. If the consumer crashes before the ack/commit, the message is delivered again.

Typical result:

- duplicates are possible
- loss is less likely

Exactly-once flow

1. Producer writes messages with idempotent or transactional guarantees.
2. Consumer processes the message.
3. Consumer updates its output/state and its consumed position atomically or transactionally.
4. If the transaction succeeds, both the result and progress marker succeed together.
5. If not, both roll back together.

Typical result:

- within that processing boundary, duplicates do not create duplicate effects

Important features:

- exactly-once across external side effects like "send email" or "charge card" is much harder
- in practice, that often still requires idempotency keys, dedupe, and compensating logic

One-line contrast

- At-most-once protects against duplicates by accepting loss
 - At-least-once protects against loss by accepting duplicates
 - Exactly-once tries to avoid both, but only with more coordination and narrower guarantees
-

6. What Problem It Solves

- Primary problem solved: defining how the system behaves when delivery, processing, and acknowledgment do not happen cleanly
 - Secondary benefits: clearer correctness model, better retry policy design, safer system behavior under crashes
 - Systems impact: changes consumer design, state management, transaction boundaries, and operational cost
-

7. When to Rely on It

Use at-most-once when:

- message loss is acceptable
- duplicates are more harmful than occasional loss
- you want minimal delivery overhead

Use at-least-once when:

- message loss is not acceptable
- the consumer can be made idempotent
- retries are a normal and expected part of the system

Use exactly-once when:

- correctness requirements are strict
- you can define a tight system boundary
- the platform supports transactional or coordinated semantics
- the added complexity is justified

Strong interviewer keywords:

- "loss tolerant" -> consider at-most-once
- "retries are okay, but duplicates must be handled" -> at-least-once plus idempotency
- "financial correctness inside the stream pipeline" -> carefully discuss exactly-once

8. When Not to Use It

Avoid at-most-once when:

- missing a message is unacceptable
- the message represents critical business state

Avoid at-least-once when:

- the consumer causes irreversible non-idempotent side effects without protection

Avoid exactly-once when:

- the team is using the phrase loosely without defining the boundary
- the problem can be solved more simply with idempotent consumers
- the downstream effects include third-party systems that cannot participate transactionally

Better alternative thinking:

- prefer at-least-once plus idempotency for most business systems
 - use exactly-once only where the platform and business need truly justify it
-

9. Pros and Cons

Guarantee	Pros	Cons
At-most-once	Simple, low overhead, no duplicate retries	Can lose messages
At-least-once	Safer against loss, practical default	Duplicates are normal and must be handled
Exactly-once	Stronger correctness within a defined boundary	Highest complexity, coordination cost, and misunderstanding risk

10. Trade-offs and Common Mistakes

Trade-offs

- Loss vs duplication: at-most-once risks loss, at-least-once risks duplicates.
- Simplicity vs correctness: exactly-once is stronger but harder and more expensive.
- Broker guarantees vs business guarantees: even strong broker semantics do not automatically make external side effects exactly once.
- Retry safety vs operational complexity: at-least-once is very practical when consumers are safe to re-run.

Common Mistakes

Mistake	Why it is wrong	Better approach
Saying "exactly-once" without defining system boundaries	The phrase becomes meaningless without scope	State where the guarantee starts and ends
Trusting the broker alone to prevent duplicate side effects	Crashes can happen after side effects but before ack/commit	Make consumers idempotent
Using at-most-once for important business events	A single transient failure can permanently lose work	Use at-least-once or stronger safeguards
Auto-committing before processing in a critical consumer	Messages can be lost if the consumer crashes mid-processing	Commit after successful processing
Assuming duplicates are rare enough to ignore	They eventually happen in production	Design for them intentionally

11. Key Numbers

These are system design heuristics.

- Retry attempts: often 3-5 before DLQ, depending on business criticality
- Visibility timeout: typically longer than expected processing duration
- Consumer processing timeout: should fit comfortably within visibility timeout or lease duration
- Duplicate window: can happen any time between side effect completion and ack/offset commit
- Exactly-once cost: usually increases latency and coordination overhead compared with at-least-once

Operational focus:

- at-most-once -> watch message loss
- at-least-once -> watch duplicates and retry volume
- exactly-once -> watch transaction failures, throughput impact, and boundary clarity

12. Failure Modes

Consumer crashes after processing but before ack

Problem:

- The work succeeded, but the broker does not know that.

User impact:

- duplicate delivery on restart

Mitigation:

Idempotent consumers

- dedupe records

Consumer acks before processing

Problem:

- The broker believes the work is done, then the consumer crashes.

User impact:

- silent message loss

Mitigation:

- ack only after successful processing

Partial transaction

Problem:

- Consumer updates database but fails before committing offset or ack.

User impact:

- duplicate reprocessing or inconsistent side effects

Mitigation:

- transactional processing where supported
- idempotent writes

Exactly-once illusion across external systems

Problem:

- Broker has strong semantics, but email provider or payment gateway does not participate.

User impact:

- duplicate emails or charges still possible

Mitigation:

- idempotency keys
- outbox/inbox patterns
- compensating logic

13. Scenario

- Product / system: Order processing platform
- At-most-once use case: low-value debug telemetry from services where some loss is acceptable
- At-least-once use case: `OrderCreated` events consumed by notification and inventory systems
- Exactly-once use case: a stream-processing stage that updates an internal ledger table and commits offsets transactionally

14. Code Sample

At-most-once style consumer mistake

```
public void consume(Record record) {
    consumer.commit(record.offset());
    process(record); // Crash here means the message is lost.
}
```

At-least-once style consumer

```
public void consume(Record record) {
    process(record);
    consumer.commit(record.offset()); // Crash before commit means duplicate delivery.
}
```

Exactly-once style sketch

```
public void consumeTransactionally(Record record) {
    transaction.begin();
    process(record);
    stateStore.save(record.key(), "PROCESSED");
    consumer.commitOffsetInTransaction(record.partition(), record.offset());
    transaction.commit();
}
```

Key idea:

- at-most-once commits early and risks loss
- at-least-once commits late and risks duplicates
- exactly-once coordinates business state and progress together

15. Mini Program / Simulation

This mini program shows the difference in failure behavior.

```
def at_most_once(process_ok: bool) -> str:
    committed = True
    if not process_ok:
        return "message lost after early commit"
    return "processed once"

def at_least_once(process_ok: bool) -> str:
    committed = False
    if not process_ok:
```

```

        return "will be retried because commit never happened"
    committed = True
    return f"processed once, commit={committed}"

def main() -> None:
    print("At-most-once failure:", at_most_once(process_ok=False))
    print("At-least-once failure:", at_least_once(process_ok=False))
    print("At-least-once success:", at_least_once(process_ok=True))

if __name__ == "__main__":
    main()

```

What this demonstrates:

- early commit creates loss risk
- late commit creates duplicate risk
- semantics are mostly about where failure is allowed to hurt you

16. Practical Question

You are designing an event consumer for order processing. Duplicate events are acceptable if the business effect remains correct, but losing events is not acceptable. Which delivery guarantee would you choose and why?

17. Strong Answer

I would choose at-least-once delivery and make the consumer idempotent.

That is the most practical production choice because it protects against message loss, which is usually the bigger business risk, while accepting that duplicates can occur during crashes and retries. I would only acknowledge or commit after successful processing, and I would make downstream writes safe to repeat by using idempotency keys, dedupe tables, or upsert semantics.

I would be careful not to promise exactly-once unless I can define a narrow boundary where both state updates and progress tracking are committed transactionally. For most business systems, at-least-once plus idempotent consumers is the right balance between correctness and complexity.

18. Revision Notes

- One-line summary: At-most-once risks loss, at-least-once risks duplicates, and exactly-once requires stronger coordination within a defined boundary.
- Three keywords: ack, retry, boundary
- One interview trap: promising exactly-once everywhere
- One memory trick: early commit loses, late commit duplicates

Topic 3: Idempotent Consumers

Track: 1.7 Asynchronous & Event-Driven Systems Scope: duplicate-safe processing, dedupe keys, upserts, inbox tables, external side effects

1. Intuition

An idempotent consumer behaves like a light switch.

If the light is already ON, pressing ON again does not change the final state. You can repeat the action, but the outcome remains the same.

That is what we want from asynchronous consumers:

- if the same message is processed again, the business result should still be correct

Short memory trick:

- same input repeated -> same final effect
-

2. Definition

- Definition: An idempotent consumer is a message or event consumer that can process the same input multiple times without causing an incorrect duplicate business effect.
 - Category: Reliability and correctness pattern
 - Core idea: retries and duplicate deliveries are normal, so the consumer must make repeated processing safe.
-

3. Why It Exists

At-least-once delivery means duplicates are expected.

Even without broker duplicates, repeats can happen because of:

- producer retries
- consumer crashes before ack
- network failures
- replays from streams
- manual reprocessing

Without idempotent consumers:

- emails may be sent twice
- inventory may be reduced twice

balances may be updated twice

- orders may be duplicated

4. Reality

Idempotent consumers are common in:

- payment processing
- order and inventory workflows
- event-driven microservices
- CDC pipelines
- replayable event streams
- webhook receivers

Common implementation styles:

- dedupe table keyed by message ID or event ID
- upsert instead of insert
- state transition guard
- unique constraint on business key
- inbox table pattern
- external idempotency key for third-party APIs

Real-world truth:

- if you use at-least-once delivery, you almost always need idempotent consumers

5. How It Works

Dedupe-record approach

1. Message arrives with a unique ID.
2. Consumer checks whether that ID was already processed.
3. If yes, it exits safely.
4. If no, it performs business logic.
5. It records the message ID as processed.
6. Then it acknowledges the message.

Upsert approach

1. Message carries a business key such as `order_id`.
2. Consumer writes using insert-or-update semantics.
3. Reprocessing the same message does not create another business record.

State-transition guard approach

1. Consumer checks current state.
2. If the state already reflects the event, it skips.
3. If not, it applies the transition once.

Important rule:

- the dedupe decision and business write should be in the same transaction when possible

6. What Problem It Solves

- Primary problem solved: making duplicate deliveries safe
- Secondary benefits: safer retries, easier replay, more robust recovery after crashes
- Systems impact: changes data model design, database constraints, consumer transaction flow, and side-effect handling

7. When to Rely on It

Use idempotent consumers when:

- the system is at-least-once
- retries are enabled
- stream replay is possible
- manual reprocessing may happen
- the business effect is costly or dangerous to duplicate

Strong interviewer keywords:

- duplicate deliveries
- retries
- replay
- exactly-once is too expensive
- webhook receiver

8. When Not to Use It

Do not skip idempotency just because:

- duplicates seem rare
- the broker claims strong guarantees
- the workflow "usually works"

The real question is not whether to use idempotency at all, but how heavy the mechanism should be.

Lighter-weight cases:

- low-value metrics might accept duplication

Heavier-weight cases:

- payments, orders, credits, inventory reservations, and external side effects need strong protection

9. Pros and Cons

Pros	Cons
Makes retries safe	Adds storage and logic overhead
Enables replay and reprocessing	Requires unique identifiers or stable business keys
Reduces business corruption from duplicates	Can be tricky around external side effects
Works well with at-least-once systems	Needs transactional thinking to avoid race conditions

10. Trade-offs and Common Mistakes

Trade-offs

- Simplicity vs correctness: dedupe logic adds complexity, but prevents business corruption.
- Storage vs safety: processed-message tables cost storage, but buy strong duplicate protection.
- Generic dedupe key vs business key: message ID is simple, but business-key idempotency can be more meaningful.
- Internal safety vs external side effects: your database may be safe, but third-party actions still need idempotency handling too.

Common Mistakes

Mistake	Why it is wrong	Better approach
Checking processed IDs outside the business transaction	Two concurrent consumers can both pass the check	Keep dedupe and business write atomic
Relying only on in-memory dedupe	Restarts lose memory	Persist dedupe state
Using non-stable identifiers	Retries may get a different key and bypass dedupe	Use stable event/message/business IDs
Making the DB write idempotent but not the external API call	Duplicate emails or charges can still happen	Use idempotency keys with external systems too

Assuming replay should not happen in production	Streams and recovery often require replay	Design for replay from day one

11. Key Numbers

These are design heuristics.

- Dedupe retention: often aligned with maximum replay or retry window
- Processed-message table size: grows with throughput x retention period
- Unique key design: should be stable across retries and replays
- External idempotency window: usually depends on provider support and business retry behavior

Operational focus:

- duplicate processing rate
- dedupe hit rate
- growth of processed-message state

12. Failure Modes

Duplicate delivery after crash

Problem:

- Consumer completes work, crashes before ack, and sees the same message again.

User impact:

- duplicate business effect if the consumer is not idempotent

Mitigation:

- processed-message table
- unique constraints
- transactional write plus dedupe

Race condition between duplicate consumers

Problem:

- Two consumers process the same logical event concurrently.

User impact:

- both may apply the state change if dedupe is not atomic

Mitigation:

- transactional dedupe
- unique constraints

External side-effect duplication

Problem:

- Internal state is protected, but email or payment provider receives the request twice.

User impact:

- duplicate email or charge

Mitigation:

- provider idempotency key
- outbox pattern
- reconciliation

13. Scenario

- Product / system: Payment and order confirmation service
- Problem: A payment-authorized event may be delivered twice after a consumer restart.
- Good design: Store the event ID in a processed-events table inside the same transaction that marks the order as paid.
- Why it works: Reprocessing the same event will hit the dedupe record and skip the duplicate update.
- What would go wrong otherwise: the order may be marked paid twice, the customer may get two emails, or downstream records may duplicate

14. Code Sample

Idempotent consumer with processed-events table

```
@Transactional
public void consume(PaymentAuthorizedEvent event) {
    if (processedEventRepository.existsById(event.eventId())) {
        return;
    }

    orderRepository.markPaid(event.orderId(), event.paymentId());
    processedEventRepository.save(new ProcessedEvent(event.eventId()));
}
```

Upsert-style idempotent write

```
INSERT INTO inventory_reservations(order_id, sku, quantity)
VALUES (:order_id, :sku, :quantity)
ON CONFLICT (order_id, sku) DO NOTHING;
```

External Side Effect with Idempotency Key

```
public void sendEmailOnce(Order order, String eventId) {
    emailClient.send(
        order.customerEmail(),
        "order-confirmation",
        eventId
    );
}
```

Key idea:

- dedupe key + atomic write is the heart of idempotency

15. Mini Program / Simulation

This mini program shows duplicate-safe processing with a processed-event set.

```
processed_events = set()
paid_orders = set()

def consume_payment_event(event_id: str, order_id: str) -> None:
    if event_id in processed_events:
        print("Duplicate ignored:", event_id)
        return

    paid_orders.add(order_id)
    processed_events.add(event_id)
    print("Processed:", event_id, "for", order_id)

def main() -> None:
    consume_payment_event("evt-101", "ord-9")
    consume_payment_event("evt-101", "ord-9")
    print("Paid orders:", paid_orders)

if __name__ == "__main__":
    main()
```

What this demonstrates:

- duplicate deliveries are harmless when the consumer tracks processed IDs
- idempotency is about final business correctness, not just avoiding code re-execution

16. Practical Question

Your queue provides at-least-once delivery, and your `PaymentAuthorized` consumer sometimes sees duplicate messages after retries. How would you make the consumer safe?

17. Strong Answer

I would make the consumer idempotent because duplicate delivery is expected in an at-least-once system. The cleanest design is to use a stable event ID and store it in a processed-events table inside the same transaction as the business update. When the event arrives, the consumer checks whether that event ID has already been applied. If yes, it skips safely. If not, it performs the state change and records the event as processed atomically.

If the workflow also triggers external side effects like email or payment operations, I would use idempotency keys with those external systems as well. The goal is not to prevent duplicate delivery at the broker layer, but to make duplicate processing harmless at the business layer.

18. Revision Notes

- One-line summary: Idempotent consumers make retries and duplicate deliveries safe by ensuring the same message cannot corrupt business state twice.
 - Three keywords: dedupe, upsert, transaction
 - One interview trap: checking dedupe outside the business transaction
 - One memory trick: same event, same final state
-

Topic 4: Dead-letter Queues

Track: 1.7 Asynchronous & Event-Driven Systems Scope: poison messages, retry exhaustion, isolation, debugging, operational recovery

1. Intuition

A dead-letter queue is the quarantine room for bad messages.

If a message keeps failing and normal retries are not helping, we stop letting it block healthy traffic. Instead, we move it aside for inspection, debugging, and controlled recovery.

Short memory trick:

- normal queue = active work
 - DLQ = failed work that needs investigation
-

2. Definition

- Definition: A dead-letter queue is a holding queue for messages that could not be processed successfully after a configured number of retries or failure conditions.

- Core idea: isolate poison messages so they do not loop forever or block healthy processing.

3. Why It Exists

Some failures are transient, and retries help.

Some failures are persistent, such as:

- malformed payloads
- bad schema version
- missing required state
- code bug
- permanently invalid business input

Without a DLQ:

- consumers retry the same poison message endlessly
- backlog grows
- good messages get delayed
- operators have no clean recovery path

4. Reality

DLQs are common in:

- queue-based worker systems
- order and payment pipelines
- stream-processing failure paths
- webhook delivery systems
- ETL ingestion pipelines

DLQs are especially useful when:

- retry exhaustion should not block the main flow
- operators need visibility into failed payloads
- bad messages must be replayed manually after a fix

Real-world truth:

- a DLQ is not a garbage bin
- it is an operational workflow

5. How It Works

1. A consumer receives a message.

2. Processing fails.
3. The system retries according to policy.
4. After the retry threshold or failure condition is reached, the message is moved to the DLQ.
5. Healthy traffic continues through the main queue or stream.
6. Operators inspect the DLQ, fix the cause, and either replay or permanently discard the message.

Typical reasons to send to DLQ:

- max retries exceeded
- validation failure
- deserialization error
- unsupported schema
- repeated timeout or downstream failure beyond policy

Important rule:

- not every failure should go to DLQ immediately
- transient failures should usually retry first

6. What Problem It Solves

- Primary problem solved: preventing poison messages from endlessly failing in the main processing path
- Secondary benefits: easier debugging, safer recovery, reduced backlog contamination
- Systems impact: changes retry policy, observability, on-call workflows, and replay tooling

7. When to Rely on It

Use a DLQ when:

- retries can exhaust
- some messages may be permanently invalid
- poison messages must not block healthy throughput
- operators need to inspect and recover failed work

Strong interviewer keywords:

- poison message
- retry exhaustion
- malformed payload
- manual replay
- operational recovery

8. When Not to Use It

Avoid treating DLQ as the first response when:

- the failure is clearly transient and should retry normally
- the failure rate is systemic and a DLQ will only hide a major outage

Avoid bad DLQ usage patterns:

- sending messages to DLQ without alerting
- never draining or reviewing the DLQ
- using DLQ as permanent storage

Better approach:

- combine retries, backoff, observability, and DLQ together

9. Pros and Cons

Pros	Cons
Isolates poison messages from healthy traffic	Adds operational workflow and tooling needs
Prevents infinite retry loops	Can hide systemic failure if nobody monitors it
Makes debugging easier	Replay must be handled carefully to avoid repeat failure
Preserves failed payloads for investigation	Unbounded DLQ growth becomes its own problem

10. Trade-offs and Common Mistakes

Trade-offs

- Retry more vs fail fast: too many retries delay recovery; too few send transient failures to DLQ too early.
- Isolation vs hidden problems: DLQ protects throughput, but can hide large-scale breakage if alerts are weak.
- Preserve data vs replay risk: keeping failed messages is useful, but replaying blindly can repeat the same outage.

Common Mistakes

Mistake	Why it is wrong	Better approach
---------	-----------------	-----------------

No DLQ for a critical queue	Poison messages can block the system indefinitely	Add retry policy and DLQ
Infinite retries with no cap	Wastes compute and starves healthy traffic	Use bounded retries then DLQ
DLQ without alerts and dashboards	Failures disappear silently	Monitor DLQ depth and age
Replaying DLQ messages blindly	The same poison message may fail again	Fix root cause first, then replay carefully
Using one shared DLQ for unrelated systems without metadata	Recovery becomes confusing	Include source queue, reason, and timestamps

11. Key Numbers

These are practical heuristics.

- Max retry count: often 3-5 attempts before DLQ
- Backoff: exponential backoff with jitter is common
- DLQ monitoring: depth, rate of arrival, and age of oldest failed message are key signals
- Replay batch size: should be controlled to avoid another failure storm

Operational focus:

- how fast DLQ fills
- why messages are landing there
- whether replay succeeds after remediation

12. Failure Modes

Poison message loops forever

Problem:

- A bad payload keeps failing and reappearing in the main queue.

User impact:

- backlog grows, healthy work slows

Mitigation:

- retry cap
- DLQ

DLQ silently fills

Problem:

Messages are being quarantined, but no alert fired

User impact:

- lost business processing and unnoticed degradation

Mitigation:

- DLQ alarms
- dashboards
- on-call runbooks

Bad replay after fix

Problem:

- Messages are replayed too aggressively or before the root cause is fixed.

User impact:

- repeated failures, another spike, duplicate side effects

Mitigation:

- replay in controlled batches
- validate fix first
- keep idempotency protections

DLQ becomes permanent graveyard

Problem:

- nobody owns cleanup or replay

User impact:

- unresolved business failures accumulate

Mitigation:

- ownership, runbooks, retention policy, and periodic review

13. Scenario

- Product / system: Order email worker
- Problem: A malformed email payload causes one message to fail every time.
- Good design: Retry a few times for transient issues, then move the message to a DLQ with error reason and metadata.
- Operational recovery: Fix the payload bug or consumer code, then replay the quarantined message safely.
- What would go wrong otherwise: the worker may keep retrying forever and delay healthy confirmations

14. Code Sample

Queue consumer with retry and DLQ

```
public void consume(OrderEmailJob job) {
    try {
        emailService.send(job.orderId(), job.email());
        queueClient.ack(job.messageId());
    } catch (TransientException ex) {
        queueClient.retry(job.messageId());
    } catch (Exception ex) {
        deadLetterQueueClient.publish(job.messageId(), job, ex.getMessage());
        queueClient.ack(job.messageId());
    }
}
```

DLQ record shape

```
{
  "sourceQueue": "order-email-jobs",
  "messageId": "msg-101",
  "failedAt": "2026-04-07T09:30:00Z",
  "reason": "invalid_email_format",
  "payload": {
    "orderId": "ord-9",
    "email": "broken-address"
  }
}
```

Replay sketch

```
public void replay(DeadLetterRecord record) {
    mainQueueClient.publish(record.payload());
    deadLetterQueueClient.markReplayed(record.messageId());
}
```

Key idea:

- DLQ isolates failure, but replay still needs discipline and idempotency

15. Mini Program / Simulation

This mini program shows a simple retry-to-DLQ flow.

```
main_queue = ["good-1", "bad-1", "good-2"]
dlq = []
attempts = {}
```

```
def process(message: str) -> bool:
    return not message.startswith("bad")

def consume_all(max_retries: int) -> None:
    for message in main_queue:
        attempts[message] = attempts.get(message, 0) + 1
        if process(message):
            print("Processed:", message)
        elif attempts[message] >= max_retries:
            dlq.append(message)
            print("Moved to DLQ:", message)
        else:
            print("Will retry:", message)

def main() -> None:
    consume_all(max_retries=2)
    consume_all(max_retries=2)
    print("DLQ contents:", dlq)

if __name__ == "__main__":
    main()
```

What this demonstrates:

- transient retry happens first
- permanently failing work is isolated after the threshold
- DLQ protects the main flow from poison messages

16. Practical Question

You run a queue for invoice-generation jobs. One malformed message keeps failing and causing worker churn. How would you design retries and dead-letter queue handling for this system?

17. Strong Answer

I would use bounded retries with exponential backoff and then move exhausted failures to a dead-letter queue.

The reason is that not every failure is transient. If one malformed or poison message keeps retrying forever, it wastes compute and delays healthy work. So I would retry a few times for transient issues like network failures, but after a configured threshold I would move the message to the DLQ along with source metadata, timestamps, and error reason.

I would also monitor DLQ depth and age, alert on abnormal growth, and provide a controlled replay workflow. Replay should only happen after the root cause is fixed, and consumers should still be idempotent so reprocessing remains safe.

18. Revision Notes

- One-line summary: A DLQ isolates messages that repeatedly fail so they stop harming the main processing flow and can be investigated safely.
- Three keywords: poison, quarantine, replay
- One interview trap: treating DLQ as a dump instead of an operational recovery mechanism
- One memory trick: retries are treatment, DLQ is quarantine

Topic 5: Backpressure Handling

Track: 1.7 Asynchronous & Event-Driven Systems Scope: overload protection, flow control, bounded buffers, consumer lag, load shedding

1. Intuition

Think of a highway exit during rush hour.

Cars can arrive faster than the road ahead can absorb them. If we keep allowing cars to enter at full speed, traffic spills backward, everything slows down, and eventually the whole area jams.

Backpressure is the system equivalent of saying:

- slow down
- wait
- reject extra work
- or process in smaller controlled batches

Short memory trick:

- no backpressure = "keep accepting until the system breaks"
- backpressure = "protect the system by matching intake to processing capacity"

2. Definition

- Definition: Backpressure is a set of mechanisms that prevent producers from overwhelming brokers, queues, consumers, or downstream services when processing capacity is lower than incoming load.
- Category: Flow control and overload protection in asynchronous systems
- Core idea: When demand exceeds capacity, the system must slow intake, buffer safely, or shed load instead of letting backlog grow without control.

In practice, backpressure can mean:

- producer throttling

Bounded queues

- pull-based consumption
- consumer pause/resume
- concurrency limits
- rate limiting
- load shedding

3. Why It Exists

Asynchronous systems do not remove capacity limits.

They only decouple producer timing from consumer timing.

That helps, but if producers permanently outrun consumers, the backlog keeps growing until one of these happens:

- memory pressure
- broker saturation
- storage growth
- latency explosion
- timeout storms
- retry amplification
- cascading failures in downstream dependencies

Backpressure exists because healthy systems must fail in a controlled way under overload, not in a chaotic way.

4. Reality

Backpressure shows up in:

- Kafka consumers falling behind during traffic spikes
- notification pipelines during flash sales
- log ingestion systems when storage or indexing is slow
- stream processors when one operator becomes slower than upstream
- webhook delivery systems when third-party endpoints are slow
- worker queues where database writes become the bottleneck

Real-world architecture truth

Backpressure is rarely one single feature.

Strong systems usually combine:

- bounded queues

consumer lag monitoring

- producer throttling
- autoscaling
- load shedding
- retries with backoff
- priority queues for important work

Another important truth:

- adding a queue is not the same as solving overload

A queue can absorb bursts, but it cannot absorb unlimited mismatch forever.

5. How It Works

The basic loop is:

1. Measure saturation.
2. Detect that consumers or downstream systems are falling behind.
3. Reduce intake or slow producers.
4. Protect the system with buffering limits or shedding.
5. Recover gradually when pressure drops.

Common backpressure signals

- queue depth
- age of oldest message
- stream consumer lag
- worker utilization
- thread-pool saturation
- rising p95 or p99 latency
- high retry rates
- growing timeout rates

Common backpressure actions

- block the producer temporarily
- return `429 Too Many Requests` or another retryable signal
- reduce concurrency
- pause consumption on hot partitions
- batch work more efficiently
- shed low-priority traffic
- spill to durable storage if the design supports it

Three important patterns

Bounded buffering

Allow a queue or in-memory buffer to grow only to a safe limit.

If that limit is hit, do not keep accepting work blindly.

Pull-based flow control

Consumers pull only as much work as they can handle.

This is healthier than unbounded push when workloads vary a lot.

Load shedding

When the system is already overloaded, it can be better to reject low-priority work early rather than degrade every request and every queue.

6. What Problem It Solves

- Primary problem solved: prevents overload from turning into uncontrolled backlog growth and cascading failure
- Secondary benefits: protects latency, preserves critical traffic, improves stability under bursts, avoids memory exhaustion
- Systems impact: determines whether async pipelines stay healthy during spikes or collapse under accumulated lag

Backpressure is fundamentally about survivability under mismatch between arrival rate and processing rate.

7. When to Rely on It

Use explicit backpressure handling when:

- producers can burst much faster than consumers
- downstream dependencies are slower or more fragile than upstream producers
- queues or streams are part of the critical path
- lag and latency matter operationally
- you need graceful degradation rather than full collapse
- workloads have different priorities and not all work is equally valuable

Especially important for:

- event ingestion systems
- stream processing systems
- notification pipelines

- payment-adjacent async workflows
- systems with many fan-in producers

8. When Not to Use It

Do not overengineer backpressure when:

- throughput is tiny and predictable
- the workload is simple enough for synchronous handling
- the queue is only smoothing very small bursts and is heavily overprovisioned

But even in smaller systems, avoid these dangerous assumptions:

- "the queue will absorb everything"
- "autoscaling alone is enough"
- "we can retry forever"

Better alternatives in very small systems:

- simple rate limiting
- a small bounded worker pool
- returning a clear retry-later response instead of buffering indefinitely

9. Pros and Cons

Pattern	Pros	Cons
Backpressure handling	Protects system stability, contains overload, preserves critical paths	Can reduce throughput, reject work, and adds operational tuning complexity

10. Trade-offs and Common Mistakes

Trade-offs

- Throughput vs latency: aggressive buffering may preserve throughput temporarily but can destroy latency.
- Availability vs quality: rejecting low-priority work can protect important traffic.
- Simplicity vs control: one queue is simple, but layered backpressure gives better survivability.
- Buffering vs shedding: buffering helps bursts; shedding helps sustained overload.

Common Mistakes

Mistake	Why it is wrong	Better approach
Using unbounded queues or buffers	Hides overload until memory or latency explodes	Use bounded queues with clear policies
Depending only on autoscaling	Scaling is not instant and may not fix a slow dependency	Combine scaling with throttling and admission control
Retrying aggressively during overload	Retries amplify pressure and can create storms	Use exponential backoff and retry budgets
Treating all traffic equally	Low-value work can crowd out critical work	Use priority classes or separate queues
Detecting pressure too late	Backlog and lag are already expensive by the time users feel pain	Alert on leading indicators such as lag and queue age

11. Key Numbers

These are practical heuristics, not universal rules.

- Queue depth: should remain within a known safe bound, not "grow until disk is full"
- Age of oldest message: often more useful than raw count because it shows user-facing delay
- Consumer lag: one of the best signals in streaming systems
- Utilization target: many teams try to avoid running core consumers at 100 percent sustained utilization
- Retry budget: retries should be capped, not infinite
- High-water mark: threshold where throttling or shedding begins
- Low-water mark: threshold where the system resumes normal intake

Interview shorthand:

- bursts need buffering
- sustained mismatch needs throttling or shedding

12. Failure Modes

Unbounded backlog growth

Problem:

- Producers continue at full speed while consumers remain slower for a long period.

User impact:

- long delays, stale work, possible memory or disk exhaustion

Mitigation:

- bounded queues

- admission control

- load shedding

Retry storm

Problem:

- Timeouts trigger retries from many callers while the downstream system is already overloaded.

User impact:

- pressure multiplies, causing wider failure and slower recovery

Mitigation:

- exponential backoff

- jitter

- retry limits

- circuit breakers

Hot partition or slow shard

Problem:

- One partition receives much more traffic or contains expensive work.

User impact:

- lag grows unevenly, ordering-sensitive consumers fall behind

Mitigation:

- better partition key design

- sharding hot keys

- workload-aware scaling

Downstream dependency collapse

Problem:

- The queue workers are healthy, but the database or external API becomes slow.

User impact:

- workers pile up, backlog grows, latency spikes across the async pipeline

Mitigation:

- concurrency caps

- fallback behavior

- degrade non-critical work

- isolate dependencies

13. Scenario

- Product / system: Ticketing platform during a major concert release
- Traffic pattern: purchase events and notification jobs spike 20x within minutes
- Problem: the notification workers and downstream email provider cannot keep up
- Good design: queue notifications, cap worker concurrency, prioritize purchase-confirmation emails over promotional emails, and shed low-value work when backlog crosses thresholds
- What would go wrong without backpressure: backlog grows without bound, retries amplify the spike, and even high-priority customer confirmations become delayed

14. Code Sample

Simple admission control pseudocode

```
public PublishResult publish(NotificationJob job) {
    int backlog = queueClient.depth("notification-jobs");

    if (backlog > HIGH_WATER_MARK && !job.isCritical()) {
        return PublishResult.rejected("retry-later");
    }

    queueClient.send(job);
    return PublishResult.accepted();
}
```

Consumer-side pause example

```
public void pollAndProcess() {
    if (databaseClient.isOverloaded()) {
        streamClient.pause("order-events");
        return;
    }

    for (EventRecord record : streamClient.poll()) {
        process(record);
    }
}
```

Key idea:

- backpressure is not only about consumers reading slower
- it is also about producers and brokers behaving safely under pressure

15. Mini Program / Simulation

This mini program shows a bounded queue with shedding when producers outrun consumers.

```

from collections import deque

queue = deque()
MAX_QUEUE = 5
accepted = 0
rejected = 0

def produce(job_id: int) -> None:
    global accepted, rejected
    if len(queue) >= MAX_QUEUE:
        rejected += 1
        print("Rejected:", job_id)
        return
    queue.append(job_id)
    accepted += 1
    print("Accepted:", job_id)

def consume(max_items: int) -> None:
    for _ in range(min(max_items, len(queue))):
        print("Processed:", queue.popleft())

def main() -> None:
    for job_id in range(1, 11):
        produce(job_id)
        if job_id % 3 == 0:
            consume(max_items=1)

    print("Remaining queue:", list(queue))
    print("Accepted:", accepted)
    print("Rejected:", rejected)

if __name__ == "__main__":
    main()

```

What this demonstrates:

- bounded buffering protects the system
- not all incoming work can be accepted safely
- under sustained mismatch, rejection is healthier than unbounded growth

16. Practical Question

Your notification pipeline receives a 30x spike during a flash sale. Queue depth, consumer lag, and retry rate are all rising. How would you design backpressure handling so the system stays stable and important notifications still go through?

17. Strong Answer

I would treat this as an overload-management problem, not just a scaling problem.

First, I would make the queues and worker pools bounded so we cannot grow backlog without limit. Then I would define high-water and low-water thresholds based on queue depth, queue age, and downstream latency. When the system crosses those thresholds, I would throttle producers, reduce or pause intake for lower-priority work, and start shedding non-critical traffic such as promotional notifications.

I would also separate priority classes so critical events like payment confirmations are isolated from best-effort traffic. On the consumer side, I would cap concurrency to protect the database or third-party providers from collapse. Retries would use exponential backoff with jitter, because aggressive retries during overload usually make the system worse.

If the workload is long-lived, I would combine this with autoscaling, but I would not depend on autoscaling alone because it reacts too slowly for sudden spikes. The goal of backpressure is graceful degradation: keep the important path alive and prevent the async system from failing everywhere at once.

18. Revision Notes

- One-line summary: Backpressure protects an async system by slowing intake, buffering safely, or shedding work when producers outrun consumers.
- Three keywords: throttling, bounded, overload
- One interview trap: believing a queue alone solves overload forever
- One memory trick: if the drain is slow, do not keep opening the tap

Topic 6: Event-Driven Architecture

Track: 1.7 Asynchronous & Event-Driven Systems Scope: decoupled services, event publication, subscribers, eventual consistency, fan-out

1. Intuition

Think of a hotel front desk.

When a guest checks in, the front desk does not directly walk over and tell every other team what to do. Instead, the check-in becomes a fact:

- room service may react
- billing may react
- analytics may react
- housekeeping may react

The front desk owns the check-in action. Other teams react to the fact that it happened.

That is the heart of event-driven architecture:

- a service publishes that something happened
- other services subscribe and react independently

Short memory trick:

- command = "do this"
- event = "this happened"

2. Definition

- Definition: Event-driven architecture is a system design style where components communicate primarily by publishing and reacting to events that describe state changes or business facts.
- Category: Distributed architecture and service-decoupling pattern
- Core idea: Producers emit events without tightly coupling themselves to all downstream consumers that may need to react.

Important distinction:

- commands are requests for work
- events are records of facts that already happened

3. Why It Exists

In tightly coupled systems, one service often calls many others directly.

That creates:

- temporal coupling
- dependency chains
- slower request paths
- brittle change management
- limited fan-out

Event-driven architecture exists because many systems need:

- one business action to trigger many downstream reactions
- independent teams to build consumers without changing the producer
- better decoupling between write paths and side effects
- durable event history or replay in some cases

It is especially attractive when the system is growing in both scale and organizational complexity.

4. Reality

Event-driven architecture is common in:

- e-commerce order workflows
- user signup and onboarding pipelines
- payment and fraud platforms
- logistics and shipment tracking
- notifications and recommendation systems
- audit and analytics pipelines

Typical technologies:

- Kafka
- Pulsar
- Kinesis
- SNS plus SQS fan-out
- EventBridge
- RabbitMQ pub/sub patterns

Real-world architecture truth

Most strong systems are hybrid, not purely event-driven.

They often use:

- synchronous APIs for immediate reads, validation, and user-facing confirmation
- events for downstream side effects, fan-out, and cross-domain propagation

Another truth:

- event-driven architecture improves decoupling, but it makes debugging, ordering, and consistency reasoning harder

5. How It Works

A common flow looks like this:

1. A user or upstream service sends a command.
2. The owning service validates the request and changes its own state.
3. That service publishes an event describing the completed business fact.
4. A broker distributes the event to interested consumers.
5. Each consumer processes the event independently and updates its own state.
6. The system reaches eventual consistency across domains.

Example

1. Checkout service creates an order.
2. It publishes `OrderPlaced`.

of inventory reserves stock.

4. Notifications send confirmation.
5. Analytics records conversion data.
6. Fraud service starts risk checks.

Important design rule

The service that owns the business state should emit the event.

Consumers should react to events, not reach back and mutate the producer's internal state directly.

Important reliability rule

If state is committed but the event is not published, the architecture breaks.

That is why teams often use:

- transactional outbox
- idempotent consumers
- schema versioning
- retries and DLQs

6. What Problem It Solves

- Primary problem solved: decouples one business action from many downstream reactions
- Secondary benefits: easier fan-out, extensibility, team independence, replay-friendly workflows, async scalability
- Systems impact: shifts systems from direct call chains toward loosely coupled event propagation and eventual consistency

Event-driven architecture is most valuable when many parts of the organization need to react to the same business facts without tight coordination.

7. When to Rely on It

Use event-driven architecture when:

- one state change must trigger multiple downstream behaviors
- producer and consumers should evolve independently
- side effects do not need to finish before the user gets a response
- eventual consistency is acceptable
- auditability or replay matters
- new consumers are likely to be added over time

Strong fits:

- order processing

Encapsulating workflows

- analytics propagation
- notifications
- inventory updates
- domain integration between microservices

8. When Not to Use It

Avoid event-driven architecture when:

- the workflow must complete synchronously before the response is considered successful
- strict cross-service transactions are required everywhere
- the system is small enough that direct calls are simpler and clearer
- the team is not ready to operate async debugging, retries, idempotency, and schema evolution

Bad reasons to use it:

- "microservices should always be event-driven"
- "queues automatically make architecture clean"
- "async is always more scalable"

Better alternatives for simpler cases:

- direct request-response calls
- a modular monolith with clear boundaries
- a single service handling the workflow end to end

9. Pros and Cons

Pattern	Pros	Cons
Event-driven architecture	Strong decoupling, natural fan-out, extensibility, async scaling	Harder debugging, eventual consistency, more operational and schema-management complexity

10. Trade-offs and Common Mistakes

Trade-offs

- Decoupling vs observability: fewer direct dependencies, but harder end-to-end tracing.
- Scalability vs consistency simplicity: async flows scale well, but reasoning about current state becomes harder.
- Extensibility vs governance: new consumers are easy to add, but event contracts need discipline.

Fast response to delayed completion does: facing latency improved, but downstream side effects may finish later.

Common Mistakes

Mistake	Why it is wrong	Better approach
Treating events like commands	Events should describe facts, not hidden remote procedure calls	Model events as past-tense business facts
Publishing without reliable state-to-event coordination	State may commit while the event is lost	Use outbox or equivalent durable publish pattern
Skipping idempotency	Consumers can see duplicates	Make consumers safe to retry
Ignoring schema evolution	New producers can break old consumers	Version schemas and evolve compatibly
Making everything asynchronous	Some flows truly need immediate synchronous confirmation	Keep a hybrid model where appropriate

11. Key Numbers

These are practical heuristics, not universal laws.

- Event size: keep events focused and reasonably small rather than embedding huge payloads
- End-to-end async SLA: define how quickly downstream consumers are expected to react
- Consumer lag: a core health signal for event-driven systems
- Retry count: bounded retries before DLQ or quarantine
- Event retention: depends on replay and audit needs
- Outbox publish delay: should remain small enough that business propagation feels timely

Interview shorthand:

- event-driven systems trade direct coupling for eventual consistency and operational discipline

12. Failure Modes

Lost publish after state commit

Problem:

- The service updates its database but crashes before publishing the event.

User impact:

- downstream systems never react, creating inconsistent cross-service state

Mitigation:

transactional context

- reliable publisher
- replay tooling

Duplicate event handling

Problem:

- A consumer processes the event, crashes, and then sees it again after retry.

User impact:

- duplicate emails, duplicate side effects, corrupted counters

Mitigation:

- idempotent consumers
- dedupe keys
- safe retries

Breaking schema change

Problem:

- A producer changes the event format in an incompatible way.

User impact:

- downstream consumers fail or silently misread data

Mitigation:

- backward-compatible schema evolution
- contract testing
- versioning

Event feedback loop

Problem:

- One event triggers another service, which emits a new event that unintentionally triggers the first workflow again.

User impact:

- event storms, duplicate workflows, runaway processing

Mitigation:

- explicit workflow boundaries
- idempotency
- correlation IDs
- loop-prevention rules

13. Scenario

- Product / system: E-commerce marketplace
- Trigger: a user places an order
- Event: `OrderPlaced`
- Downstream consumers: inventory, analytics, notification, fraud, loyalty points, audit
- Why event-driven architecture is good here: the order service should not synchronously call every downstream team before returning success
- What would go wrong with only direct calls: tight coupling, slow checkout path, cascading failures, and painful onboarding of new consumers

14. Code Sample

Producer-side pseudocode with outbox idea

```
public void placeOrder(CreateOrderCommand command) {
    Order order = orderRepository.save(Order.create(command));
    outboxRepository.save(new OutboxEvent (
        "OrderPlaced",
        order.id().toString(),
        serialize(new OrderPlacedEvent(order.id(), order.userId(), order.total()))
    ));
}
```

Outbox publisher sketch

```
public void publishOutboxBatch(List<OutboxEvent> events) {
    for (OutboxEvent event : events) {
        eventBus.publish(event.type(), event.key(), event.payload());
        outboxRepository.markPublished(event.id());
    }
}
```

Consumer example

```
public void onOrderPlaced(OrderPlacedEvent event) {
    inventoryService.reserve(event.orderId());
    analyticsService.recordOrder(event.orderId(), event.total());
}
```

Key idea:

- the producer owns the fact
- the broker distributes the fact
- consumers react independently

15. Mini Program / Simulation

This mini program shows one event fan out to multiple independent consumers.

```
subscribers = {
    "OrderPlaced": [],
}

def subscribe(event_type: str, handler) -> None:
    subscribers.setdefault(event_type, []).append(handler)

def publish(event_type: str, payload: dict) -> None:
    for handler in subscribers.get(event_type, []):
        handler(payload)

def analytics_handler(payload: dict) -> None:
    print("Analytics saw order:", payload["order_id"])

def notification_handler(payload: dict) -> None:
    print("Notification sent for order:", payload["order_id"])

def loyalty_handler(payload: dict) -> None:
    print("Points awarded for order:", payload["order_id"])

def main() -> None:
    subscribe("OrderPlaced", analytics_handler)
    subscribe("OrderPlaced", notification_handler)
    subscribe("OrderPlaced", loyalty_handler)

    publish("OrderPlaced", {"order_id": "o-101"})

if __name__ == "__main__":
    main()
```

What this demonstrates:

- the producer does not know all consumers
- one business fact can fan out to many reactions
- new consumers can be added without changing the producer code

16. Practical Question

You are designing a signup platform. After a user account is created, the system must send a welcome email, provision a profile, emit analytics, and trigger fraud checks. How would you decide

what stays synchronous and what becomes event-driven?

17. Strong Answer

I would keep only the minimum critical path synchronous, and move independent side effects to event-driven processing.

The account service should synchronously validate the request, create the user record, and return success only after its own source-of-truth state is committed. After that, it should publish a `UserCreated` event. Consumers such as email, analytics, profile initialization, and fraud checks can react independently.

This gives me loose coupling and makes it easy to add new subscribers later without changing the signup service. It also shortens the user-facing response path because the signup API does not block on every side effect. The trade-off is eventual consistency, so I would use reliable publish semantics such as an outbox, make consumers idempotent, version event schemas carefully, and trace the workflow with correlation IDs.

I would not force everything into events. If the business requires a step to complete before signup is considered successful, that step should remain synchronous.

18. Revision Notes

- One-line summary: Event-driven architecture lets services publish business facts and lets other services react independently with loose coupling.
- Three keywords: fan-out, decoupling, eventual-consistency
- One interview trap: making everything asynchronous without deciding what truly belongs on the critical path
- One memory trick: command asks, event tells